



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

MATHEUS FELIPE DE OLIVEIRA

PROJETO CALCULADORA CIENTÍFICA

**RIO GRANDE DO NORTE - NATAL
2026**

SUMÁRIO

INTRODUÇÃO.....	3
OBJETIVOS.....	4
Objetivos gerais:.....	4
Objetivos específicos:.....	4
FUNDAMENTAÇÃO TEÓRICA.....	5
Cálculo de funções transcendentais na computação e erro de truncamento:.....	5
Formulação matemática das equações :.....	6
METODOLOGIA.....	7
Ferramentas utilizadas e tratamento de dados de entrada.....	7
Programação da calculadora e estruturas de repetição.....	7
Discretização numérica e renderização gráfica.....	8
RESULTADOS E DISCUSSÕES.....	10
Validação numérica e sua precisão.....	10
Validação da visualização gráfica.....	11
CONCLUSÃO.....	12
REFERÊNCIAS.....	13

INTRODUÇÃO

As calculadoras científicas e sistemas computacionais modernos desempenham um papel fundamental nas engenharias e ciências exatas ao viabilizarem a resolução de problemas matemáticos complexos. Contudo, a arquitetura dos computadores digitais é baseada em operações aritméticas elementares e em uma representação de memória finita. Consequentemente, dispositivos computacionais não armazenam valores pré-calculados para cada argumento contínuo de uma função transcendental.

A avaliação de funções não-algébricas, como as trigonométricas, exponenciais e logarítmicas, exige métodos de aproximação que substituam o cálculo de infinitos termos por somatórios finitos tratáveis por software. Para contornar essa limitação, a computação numérica recorre à expansão em Séries de Taylor e Maclaurin. Esse aparato matemático permite aproximar funções contínuas por meio de polinômios de grau n , cuja precisão é diretamente controlada pelo número de termos somados e pelo erro de truncamento inerente.

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)(x - x_0)^k}{k!}$$

IMAGEM 1: *Representação da Série de Taylor*

Nesse contexto, este projeto apresenta o desenvolvimento de uma calculadora científica e plotadora gráfica implementada em Python. O núcleo computacional foi construído integralmente sem o auxílio de bibliotecas matemáticas nativas ou comerciais para funções transcendentais. Todas as aproximações numéricas foram estruturadas a partir de algoritmos iterativos baseados em expansões polinomiais, unindo os fundamentos teóricos da computação numérica à programação prática.

OBJETIVOS

Objetivos gerais:

- Desenvolver uma Calculadora científica e plotadora gráfica com um núcleo matemático totalmente em Python, implementando funções transcendentais através de Séries de Taylor e Maclaurin sem o uso de bibliotecas matemáticas nativas.

Objetivos específicos:

- Modelar e implementar algoritmos para o cálculo numérico de funções transcendentais (trigonométricas, exponenciais e logarítmicas) e operações fundamentais (fatoriais e radiciação) empregando expansões por Séries de Taylor e Maclaurin.
- Integrar um módulo de renderização gráfica bidimensional capaz de plotar a curva contínua da função avaliada, garantindo o tratamento computacional de descontinuidades e destacando geometricamente a coordenada exata calculada no plano cartesiano.

FUNDAMENTAÇÃO TEÓRICA

Cálculo de funções transcendentais na computação e erro de truncamento:

Na arquitetura de computadores, as Unidades Lógicas e Aritméticas são projetadas para executar operações matemáticas elementares, como a adição, subtração, multiplicação e divisão. No entanto, o cálculo de funções transcendentais, assim como as funções trigonométricas ($\sin(x)$, $\cos(x)$), exponenciais (e^x) e logarítmicas ($\ln(x)$), exige métodos mais sofisticados, uma vez que não podem ser expressas por um número finito de operações algébricas básicas. Para que essas funções sejam processadas via software, é necessário aproximá-las por meio de polinômios, sendo os métodos de expansão em série uma das abordagens mais consolidadas no cálculo numérico.

Para isso, a base matemática utilizada para a construção do motor de cálculo desta aplicação fundamenta-se no Teorema da Série de Taylor. Esse teorema estabelece que, se uma função $f(x)$ possui derivadas contínuas até a ordem $n + 1$ em um intervalo contendo um ponto a , ela pode ser expandida em um polinômio de grau n da seguinte forma:

Para os algoritmos implementados nesta calculadora, optou-se pela expansão em torno do ponto $a = 0$, um caso particular da Série de Taylor conhecido como Série de Maclaurin. Essa escolha reduz a complexidade computacional das aproximações em torno da origem. Por exemplo, a implementação da função exponencial baseia-se na expansão:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

IMAGEM 2: Representação da Série de Maclaurin com “a” centrado em 0, na função exponencial.

Do ponto de vista analítico, a Série de Taylor converge exatamente para o valor da função quando “ $n \rightarrow \infty$ ” (dentro do raio de convergência). Porém, na computação prática, é impossível calcular infinitos termos. O algoritmo implementado no código realiza a soma até um número pré definido de iterações N .

Essa interrupção gera o chamado erro de truncamento. Quanto maior o número de termos N calculados, menor será o erro de truncamento e maior a precisão do resultado. No entanto, aumentar demasiadamente os termos implica em maior custo computacional e expõe o cálculo ao erro de arredondamento, decorrente da limitação de bits na representação de pontos flutuantes da decorrente representação em vírgula flutuante utilizada pela arquitetura dos computadores. A lógica da calculadora foi estruturada para balancear a precisão requerida e o tempo de execução iterativo.

Formulação matemática das equações :

Para a construção do núcleo da calculadora, foi necessária a implementação algorítmica de diversas funções matemáticas, desde operações básicas até funções transcendentais. As aproximações foram modeladas da seguinte forma:

Fatorial (n!): Operação base para a maioria das expansões em série, calculada de forma iterativa, onde $n! = n \cdot (n-1) \cdot (\dots) \cdot 1$

Função Exponencial: Aproximada pela Série de Maclaurin, definida pela somatória de $k = 0$ até o

limite de iterações, tal que $e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$.

Logaritmos: A expansão convencional de Taylor para logaritmos possui convergência lenta e domínio limitado. Por isso, o logaritmo natural ($\ln(x)$) foi implementado utilizando uma série otimizada, baseada na transformação $y = \frac{x-1}{x+1}$, que garante uma convergência muito mais rápida e abrange todos os reais positivos:

$$\ln(x) = 2 \sum_{n=0}^{\infty} \frac{1}{2n+1} \left(\frac{x-1}{x+1} \right)^{2n+1} = 2 \left[\left(\frac{x-1}{x+1} \right) + \frac{1}{3} \left(\frac{x-1}{x+1} \right)^3 + \dots \right]$$

A partir dele, implementou-se a mudança de base para o logaritmo decimal através da relação:

$$\log_{10}(x) = \frac{\ln(x)}{\ln(10)}$$

Funções Trigonômétricas:

- O **Seno** ($\sin(x)$) é calculado pela expansão utilizando apenas os termos ímpares da série.

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

- O **Cosseno** ($\cos(x)$) é calculado utilizando apenas os termos pares.

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

- *Nota:* Como as séries operam fundamentalmente em radianos, o algoritmo realiza previamente a conversão para entradas em graus através da relação $x_{rad} = x_{deg} \cdot \frac{\pi}{180}$.

Tangente: Optou-se pela relação trigonométrica direta $\tan(x) = \frac{\sin(x)}{\cos(x)}$.

Funções Hiperbólicas: Definidas diretamente a partir da função exponencial previamente

implementada, sendo o seno hiperbólico $\sinh(x) = \frac{e^x - e^{-x}}{2}$ e o cosseno hiperbólico $\cosh(x) = \frac{e^x + e^{-x}}{2}$.

Radiciação: Generalizada para calcular a n -ésima raiz de x manipulando a propriedade

logarítmica-exponencial: $\sqrt[n]{x} = e^{\frac{1}{n} \ln(x)}$.

METODOLOGIA

Ferramentas utilizadas e tratamento de dados de entrada

O projeto foi desenvolvido em Python, sob a restrição de não utilizar o módulo nativo “**math**” para as operações transcendentais. As bibliotecas externas “**numpy**” e “**matplotlib**” foram importadas exclusivamente para a etapa posterior de renderização gráfica. Antes de acionar o núcleo matemático, os dados inseridos pelo usuário passam por uma rotina de tratamento (função `processar_entrada`). Essa etapa normaliza as strings de texto, interpretando notações informais como “3pi” e convertendo-as para o valor numérico aproximado de π (calculado via Fórmula de Machin), garantindo que o programa processe apenas valores de ponto flutuante válidos.

```
def processar_entrada(texto):  
    PI = pi_taylor() #formula p/ calcular pi com taylor  
  
    texto = texto.lower().replace(" ", "") # limpa espaços e deixa tudo em minúsculo  
  
    texto = re.sub(r"(\d)(pi)", r"\1*\2", texto) # transforma (ex) "3pi" em "3*pi" para o python entender a multiplicação  
  
    texto = texto.replace("pi", str(PI)) # substitui 'pi' pelo valor numérico correto  
  
    try:  
        return float(eval(texto)) # no final avalia a expressão matemática tratada  
    except:  
        raise ValueError("Expressão inválida!")
```

IMAGEM 3: Parte do código referente ao processamento e tratamento de dados. Foto tirada do VS Code.

Programação da calculadora e estruturas de repetição

O núcleo da calculadora foi construído transcrevendo as Séries de Taylor e Maclaurin para funções desejadas. Como a base de grande parte das expansões em série exige denominadores com fatoriais, implementou-se primeiramente uma função iterativa (`calc_fatorial`) para sustentar as demais operações. A partir dessa base, as funções matemáticas contínuas foram modeladas utilizando estruturas de repetição (`while` ou `for`). O laço atua como um somatório, acumulando os termos do polinômio a cada iteração.

```
def fatorial(num):  
    resultado = 1  
    contador = 1  
    while contador <= num:  
        resultado = resultado * contador  
        contador = contador + 1  
    return resultado  
  
def exp_taylor(num):  
    soma = 1  
    k = 1  
    while k <= 100:  
        termo = (num**k) / fatorial(k)  
        soma = soma + termo  
        k = k + 1  
    return soma
```

IMAGEM 4: Parte do código referente às funções de “fatorial” e “exponencial”. Foto tirada do VS Code.

O critério de parada foi fixado em 100 iterações, um limite estabelecido para assegurar a convergência matemática e manter o erro de truncamento insignificante para a precisão do sistema. Lógicas semelhantes foram aplicadas para as funções trigonométricas, alternando o acúmulo entre termos pares e ímpares de acordo com as fórmulas do seno e cosseno, além da alternância entre radianos ou graus.

```
def seno_taylor(num, tipo):
    if tipo == "graus":
        num = num * (pi_taylor() / 180) # se o tipo for graus, converte num para radianos usando pi_taylor

    #reduz o ângulo para o intervalo de 0 a 2*pi para a série de taylor não explodir
    num = num % (2 * pi_taylor())

    seno = 0
    k = 0
    while k <= 15:
        n = 2 * k + 1
        termo = (((-1) ** k) * (num**n)) / fatorial(n)
        seno += termo
        k = k + 1
    return round(seno, 10)

def cosseno_taylor(num, tipo):
    if tipo == "graus":
        num = num * (pi_taylor() / 180) # se o tipo for graus, converte num para radianos usando pi_taylor

    #reduz o ângulo para o intervalo de 0 a 2*pi para a série de taylor não explodir
    num = num % (2 * pi_taylor())

    cos = 0
    k = 0
    while k <= 15:
        n = 2 * k
        termo = (((-1) ** k) * (num**n)) / fatorial(n)
        cos += termo
        k = k + 1
    return round(cos, 10)

def tangente_taylor(num, tipo):
    if tipo.lower() == "graus":
        num = num * (pi_taylor() / 180)

    tang = seno_taylor(num, "radianos") / cosseno_taylor(num, "radianos")
    return round(tang, 10)
```

IMAGEM 5: Parte do código referente às funções transcendentais, seno, cosseno e tangente. Foto tirada do VS Code.

Discretização numérica e renderização gráfica

1. Determinação do domínio / eixo X: para que o computador consiga desenhar uma linha curva suave, ele precisa calcular centenas de pontos próximos ao valor digitado pelo usuário. Para isso, foi utilizada a função “**np.linspace**”, que gera um vetor contendo um intervalo de coordenadas linearmente espaçadas no eixo horizontal.
2. Cálculo da imagem / eixo Y: O algoritmo calcula sobre cada ponto e chama a função matemática correspondente (ex: a `tangente_taylor`) para calcular o eixo Y. Como a calculadora lida com funções que possuem descontinuidades ou assíntotas, o cálculo de Y foi inserido em um bloco de tratamento de exceções (try/except). Valores inválidos ou excessivamente grandes são substituídos por “**np.nan**” (Not a Number). Isso impede que o sistema desenhe uma linha reta artificial cruzando a tela inteira quando a função "quebra".
3. Destaque do Ponto Calculado: Após desenhar a curva principal com a função “**plt.plot()**”, o par ordenado exato (x,y) que o usuário solicitou no terminal é destacado visualmente. O script utiliza “**plt.scatter()**” para plotar um marcador vermelho sobre a coordenada, e desenha

linhas pontilhadas conectando esse ponto diretamente aos eixos X e Y, facilitando a interpretação do resultado.

```
# AJUSTE INTELIGENTE DA JANELA (ZOOM) - Domínio.
if tipo_angulo == "graus":
    eixo_x = np.linspace(num_central - 360, num_central + 360, 200)
elif nome_operacao in ["ln", "log"]:
    inicio = 0.1
    fim = num_central + 20 if num_central > 0 else 20
    eixo_x = np.linspace(inicio, fim, 100)
elif nome_operacao == "potencia":
    eixo_x = np.linspace(num_central - 3, num_central + 3, 100)
else:
    eixo_x = np.linspace(num_central - 10, num_central + 10, 100)
```

IMAGEM 6: Parte do código referente aos domínios da função. Foto tirada do GitHub.

```
# Calcula o Y para cada pontinho do X e acha o Y central - Imagem da função
for x in eixo_x:
    if nome_operacao == "exponencial":
        y = exp_taylor(x)
        if x == num_central: y_central = y
    elif nome_operacao == "ln":
        y = ln_taylor(x) if x > 0 else np.nan
    elif nome_operacao == "log":
        y = log_taylor(x) if x > 0 else np.nan
    elif nome_operacao == "potencia":
        y = potencial0(x)
    elif nome_operacao == "seno":
        y = seno_taylor(x, tipo_angulo)
    elif nome_operacao == "cosseno":
        y = cosseno_taylor(x, tipo_angulo)
    elif nome_operacao == "tangente":
        y = tangente_taylor(x, tipo_angulo)
        if y > 200 or y < -200: y = np.nan
    elif nome_operacao == "senh":
        y = senh_taylor(x)
    elif nome_operacao == "cosh":
        y = cosh_taylor(x)

    eixo_y.append(y)
```

IMAGEM 7: Parte do código referente às imagens da função. Foto tirada do GitHub.

```
# Desenha o gráfico
plt.figure(figsize=(8, 5))
plt.plot(eixo_x, eixo_y, color='purple', linewidth=2, label=nome_operacao)

# Desenha a linha pontilhada marcando o X
plt.axvline(num_central, color='red', linestyle='--', alpha=0.5)

# Coloca o ponto no gráfico
if not np.isnan(y_central):
    # Desenha uma bolinha vermelha no ponto exato
    plt.plot(num_central, y_central, marker='o', color='red')

# Escreve as coordenadas (x, y) do lado da bolinha
plt.annotate(
    f'({num_central}, {round(y_central, 4)})', # Texto que vai aparecer (arredondado para 4 casas para ficar bonito)
    xy=(num_central, y_central), # Ponto de referência
    xytext=(10, 10), # Afasta o texto 10 pontinhos pra não ficar em cima da bolinha
    textcoords='offset points',
    fontsize=10,
    fontweight='bold',
    color='darkred'
)
#
```

IMAGEM 8: Parte do código referente a plotagem do gráfico da função. Foto tirada do GitHub.

RESULTADOS E DISCUSSÕES

Validação numérica e sua precisão.

Para testar a confiabilidade da calculadora científica, os resultados gerados pelos algoritmos de Séries de Taylor foram submetidos a testes usando diversos valores e funções. Para validar a acurácia do núcleo matemático construído, calculou-se o erro absoluto das operações, definido pela diferença direta entre o valor aproximado gerado pelo algoritmo e o valor de referência (obtido através da calculadora científica padrão do Windows 11). A Tabela 1 apresenta uma amostra das operações realizadas, evidenciando a precisão do método.

Função	Entrada	Resultado	Referência	Erro absoluto
Seno	90°	1	1	0
Cosseno	$\pi/3$	0,5	0,5	0
Tangente	10°	0,1763269807	0,176326980708 46497347109038 686862	$8,46 \times 10^{-11}$
Exponencial	5	148,4131591025 7657	148,4131591025 76603421115580 04055	$3,34 \times 10^{-14}$
Logaritmo Natural	10	2,302585092994 0455	2,302585092994 04568401799145 46844	$1,84 \times 10^{-16}$
Seno Hiperbólico	0	0	0	0
Cosseno Hiperbólico	0	1	1	0
Seno Hiperbólico	1	1.1752011936438 016	1,1752011936438 014	$2,00 \times 10^{-16}$
Cosseno Hiperbólico	1	1.543080634815 244	1,543080634815 243	$1,00 \times 10^{-15}$
Seno Hiperbólico	3	10.01787492740 9899	10,01787492740 99	$1,00 \times 10^{-15}$
Cosseno Hiperbólico	3	10.06766199577 7762	10,06766199577 7765	$3,00 \times 10^{-15}$
Seno Hiperbólico	5	74.20321057778 874	74,20321057778 87	$4,00 \times 10^{-14}$
Cosseno Hiperbólico	5	74.20994852478 783	74,20994852478 784	$1,00 \times 10^{-14}$

TABELA 1: Entrada e saída de valores correspondentes a suas funções.

Conforme observado nos testes, a calculadora obteve êxito ao reproduzir os valores esperados com alta acurácia, com o erro absoluto atingindo a magnitude de quadrilhonésimos (10^{-15}) em diversos casos. Isso valida a eficácia do tratamento de dados inicial, como a conversão de graus para radianos e a redução ao primeiro quadrante. Na função exponencial, por exemplo, a iteração de 100 termos no polinômio de Maclaurin mostrou-se robusta o suficiente para calcular valores elevados sem exceder o limite de tempo de execução. Já para as funções trigonométricas, o limite estipulado de $k \leq 15$ é ideal, pois, a partir desse ponto, o crescimento do fatorial no denominador torna o valor adicionado ao somatório tão ínfimo que não altera a precisão final da máquina.

Validação da visualização gráfica.

A geração gráfica foi validada usando a biblioteca “**matplotlib**”. A Figura 1 ilustra a função seno, apresentando uma curva contínua e suave graças à alta quantidade de pontos gerados pela função “**np.linspace**”. O destaque em vermelho no par ordenado comprova que o cálculo matemático e a renderização do gráfico estão perfeitamente integrados.



IMAGEM 9: Visualização gráfica da função $\text{Sen}(x)$, quando $x = \pi$.

CONCLUSÃO

Por fim, como visto pelos testes realizados, é possível comprovar a eficiência da calculadora desenvolvida em Python. Ela conseguiu cumprir seus objetivos de demonstrar a eficácia do método das Séries de Taylor e Maclaurin para aproximar funções transcendentais. Além disso, comprovou que, com o tratamento de dados e o controle de truncamento, é possível criar uma calculadora útil sem a dependência de módulos nativos da linguagem, sendo capaz, inclusive, de plotar os gráficos com alta resolução.

REFERÊNCIAS

- RUGGIERO, Márcia A. Gomes; LOPES, Vera Lúcia da Rocha. **Cálculo Numérico: aspectos teóricos e computacionais**. 2. ed. São Paulo: Makron Books, 1996.
- BURDEN, Richard L.; FAIRES, J. Douglas. **Análise Numérica**. 8. ed. São Paulo: Cengage Learning, 2008.
- FRANCO, Neide Bertoldi. **Cálculo Numérico**. 1. ed. São Paulo: Pearson Prentice Hall, 2006.
- UFRGS. Instituto de Matemática e Estatística. **Cálculo Numérico: Um Livro Colaborativo**. Porto Alegre: UFRGS, 2026. Disponível em: <https://www.ufrgs.br/reatmat/CalculoNumerico/livro-py/>. Acesso em: 28 set. 2026.